UNIT 3 – Searching and Sorting

Searching: Concept of Searching, Sequential search, Index Sequential Search, Binary Search. Concept of Hashing & Collision resolution Techniques used in Hashing.
Sorting: Insertion Sort, Selection, Bubble Sort, Quick Sort, Merge Sort, Heap Sort and Radix Sort.

◇ PART 1: SEARCHING

◇ 1. Concept of Searching

Definition:

Searching means **finding the location of a particular element (key)** in a list or array of elements.

Example:

If you have roll numbers `[10, 20, 30, 40, 50]` and you want to find `30`, the process you use to find it is called **searching**.

Two main types:

1. **Sequential (Linear) Search**
2. **Binary Search**
3. **Index Sequential Search**
4. **Hashing (Advanced technique)**

◇ 2. Sequential Search (Linear Search)

Idea:

Check each element one by one until the key is found or the list ends.

Algorithm (Array of n elements):

```
for i = 0 to n-1:
    if arr[i] == key:
        return i
return -1   // not found
```

Example:

Array = [5, 8, 2, 9, 1], Key = 9

→ check 5 (no), 8 (no), 2 (no), 9 (yes, found at position 4)

Complexity:

Case	Time Complexity
Best Case (first element)	O(1)
Worst Case (last element or not found)	O(n)
Average Case	O(n/2) ≈ O(n)

Use when:

- List is small or unsorted.

◇ 3. Binary Search

Idea:

Binary search works only on **sorted data**.

It divides the list into two halves repeatedly and eliminates one half at a time.

Algorithm:

```
low = 0
high = n-1
while low <= high:
    mid = (low + high) // 2
    if key == arr[mid]:
        return mid
    elif key < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1
return -1
```

Example:

Array = [10, 20, 30, 40, 50], Key = 40

→ mid = 30 → 40 > 30 → search right half

→ mid = 40 → found ✅

Complexity:

Case	Time Complexity
Best	O(1)
Worst	$O(\log_2 n)$
Average	$O(\log_2 n)$

Use when:

- List is sorted.
- Fast searching is needed.

Common mistake:
Students forget that **binary search only works on sorted arrays**.

◇ 4. Index Sequential Search

Idea:
A hybrid method combining **binary search on index** + **linear search on block**.

How it works:

1. The data is divided into small blocks.
2. Each block has an **index** (like a summary).
3. First, search the index using binary search to locate the correct block.
4. Then, perform linear search inside that block.

Example:
Data divided into 3 blocks:

- Block 1: 10, 15, 18
- Block 2: 22, 25, 27
- Block 3: 31, 33, 35

Index = [18, 27, 35]
If key = 26 → Binary search index → Block 2 → Linear search inside → Found.

Complexity:
O(log m + n/m)
(where *m* = number of blocks)

Use when:

- Large sorted dataset stored on disk or database.

◇ 5. Hashing

Concept:
Hashing is used for **fast data retrieval** using a **hash function** that converts a key into an **index** in a hash table.

Hash Function:
It takes a key and returns an index number.
Example:

```
index = key % table_size
```

If table size = 10 and key = 32 → index = 32 % 10 = 2

Problem:

Sometimes two keys map to the same index → this is called **collision**.

◇ *Collision Resolution Techniques*

1. **Open Addressing**
    a. All elements stored in the same table.
    b. If collision occurs, search next empty slot.

Techniques:

c. **Linear Probing:** next index = (h(key) + 1) % size
d. **Quadratic Probing:** next index = (h(key) + i^2) % size
e. **Double Hashing:** next index = (h1(key) + i*h2(key)) % size

2. **Chaining**
    a. Each table index has a linked list.
    b. Colliding elements are added to the list at that index.

Example:

Keys: [23, 43, 13, 27], Table size = 10

Hash function: h(key) = key % 10

→ 23→3, 43→3 (collision), 13→3 (collision), 27→7

→ Use chaining: at index 3 → [23 → 43 → 13]

Advantages of Hashing:

- Very fast search (O(1) average case).

Disadvantages:

- Collisions slow it down.
- Needs extra space for table or links.

◇ PART 2: SORTING

◇ 1. Concept of Sorting

Sorting means **arranging data in a particular order** — ascending or descending.

Example:

Before sort: [5, 1, 4, 2, 8]

After sort (ascending): [1, 2, 4, 5, 8]

◇ 2. Insertion Sort

Idea:

Insert elements one by one into the correct position like sorting playing cards.

Algorithm:

```
for i = 1 to n-1:
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j = j - 1
    arr[j + 1] = key
```

Example:

[5, 3, 4]

→ take 3 → insert before 5 → [3, 5, 4]

→ take 4 → insert before 5 → [3, 4, 5]

Complexity:

Case	Time
Best	O(n)
Worst	$O(n^2)$
Stable	✅ Yes

◇ 3. Selection Sort

Idea:

Find the smallest element and put it at the beginning.

Algorithm:

```
for i = 0 to n-2:
    min = i
    for j = i+1 to n-1:
        if arr[j] < arr[min]:
            min = j
    swap(arr[min], arr[i])
```

Example:

[64, 25, 12, 22] → pick smallest (12) → place front → [12, 25, 64, 22] ...

Complexity:

Always $O(n^2)$, not stable.

◇ 4. Bubble Sort

Idea:

Repeatedly compare and swap adjacent elements if they are in wrong order.

Algorithm:

```
for i = 0 to n-1:
    for j = 0 to n-i-2:
        if arr[j] > arr[j+1]:
            swap(arr[j], arr[j+1])
```

Example:

[5, 3, 4] → compare (5,3) → swap → [3,5,4] → compare (5,4) → swap → [3,4,5]

Complexity:

Case	Time
Best	O(n) (if optimized)
Worst	$O(n^2)$
Stable	✅ Yes

◇ 5. Quick Sort

Idea:

Divide and conquer — choose a **pivot**, partition array into two parts:

- Left part < pivot
- Right part > pivot

Then sort both recursively.

Algorithm:

```
quickSort(arr, low, high):
    if low < high:
        pivot = partition(arr, low, high)
        quickSort(arr, low, pivot-1)
        quickSort(arr, pivot+1, high)
```

Example:

[10, 7, 8, 9, 1, 5] pivot=5 → partition → [1] 5 [10,7,8,9]

Complexity:

Case	Time
Best	O(n log n)
Worst	$O(n^2)$
Average	O(n log n)
Stable	❌ No

◇ 6. Merge Sort

Idea:

Divide array into halves, sort each half, then merge them.

Algorithm:

```
mergeSort(arr):
    if n > 1:
        mid = n//2
        L = arr[:mid]
        R = arr[mid:]
        mergeSort(L)
        mergeSort(R)
        merge(L, R, arr)
```

Example:

[38, 27, 43, 3, 9] → split → sort → merge → [3, 9, 27, 38, 43]

Complexity:

Case	Time	Space	Stable
All	O(n log n)	O(n)	✅ Yes

◇ 7. Heap Sort

Idea:

Use a binary heap (complete binary tree) to get elements in sorted order.

Steps:

1. Build a max heap.
2. Swap root with last element.
3. Reduce heap size and heapify again.

Complexity:

O(n log n), Not stable.

◇ 8. Radix Sort

Idea:

Sort numbers digit by digit using a stable sort like Counting Sort.

Example:

[170, 45, 75, 90, 802, 24, 2, 66]

→ sort by 1's place → then 10's → then 100's → get final sorted array.

Complexity:

O(d*(n + k))

(d = number of digits, k = range of digits)

Comparison Table

Algorithm	Best	Worst	Stable	Type

Insertion Sort	O(n)	$O(n^2)$	✅	Internal
Selection Sort	$O(n^2)$	$O(n^2)$	❌	Internal
Bubble Sort	O(n)	$O(n^2)$	✅	Internal
Quick Sort	O(n log n)	$O(n^2)$	❌	Internal
Merge Sort	O(n log n)	O(n log n)	✅	External
Heap Sort	O(n log n)	O(n log n)	❌	Internal
Radix Sort	O(d*(n+k))	O(d*(n+k))	✅	Non-comparison

◇ 1. Difference between Linear Search and Binary Search

Feature	Linear (Sequential) Search	Binary Search
Data requirement	Works on **unsorted or sorted** data	Works only on **sorted** data
Method	Check each element one by one	Repeatedly divide the list into halves
Time Complexity (Worst)	O(n)	$O(\log_2 n)$
Space Complexity	O(1)	O(1)
Efficiency	Slow for large datasets	Very fast for large sorted datasets
Implementation	Simple and easy	Slightly complex
Example	Search 5 in [2,4,5,7] → check all	Search 5 in [2,4,5,7] → mid=4 → right half → found

◇ 2. Difference between Sequential Search and Index Sequential Search

Feature	Sequential Search	Index Sequential Search
Data type	Works on any list	Works on **large sorted data** divided into blocks
Search process	Linearly scans all elements	Binary search on index + Linear search in block
Speed	Slower (O(n))	Faster (O(log m + n/m))
Storage overhead	None	Requires index table
Use case	Small datasets	Databases or file systems

Example	Search directly in whole array	Search index → go to block → find element

◇ 3. Difference between Hashing and Searching

Feature	Hashing	Searching (like Binary/Linear)
Concept	Direct access using a **hash function**	Check elements one by one or by dividing
Data structure	Hash Table	Array or Linked List
Time Complexity (Average)	O(1)	O(n) or O(log n)
Precondition	Depends on hash function	Depends on data order (for binary search)
Collision	May occur, needs resolution	No collision concept
Usage	Databases, Symbol tables	General data searching

◇ 4. Difference between Internal Sorting and External Sorting

Feature	Internal Sorting	External Sorting
Where sorting happens	Entire data fits into **main memory (RAM)**	Data too large → stored on **external storage (disk)**
Examples	Bubble, Insertion, Selection, Quick, Heap	Merge Sort (External), Polyphase Merge
Speed	Faster	Slower (due to I/O operations)
Used for	Small to medium data	Very large datasets
Memory Requirement	Less	High (needs disk access)

◇ 5. Difference between Bubble Sort and Selection Sort

Feature	Bubble Sort	Selection Sort
Method	Repeatedly swaps adjacent elements	Finds smallest element and places at start

Number of Swaps	High	Low
Stability	Stable ✅	Not Stable ❌
Time Complexity	$O(n^2)$	$O(n^2)$
Best Case	O(n) (if optimized)	$O(n^2)$
Example use	When small swaps allowed	When swap cost is high

◇ 6. Difference between Quick Sort and Merge Sort

Feature	Quick Sort	Merge Sort
Method	Divide around a **pivot**	Divide into halves and **merge** sorted parts
Approach	Divide and Conquer	Divide and Conquer
Time Complexity (Avg)	O(n log n)	O(n log n)
Time (Worst)	$O(n^2)$	O(n log n)
Space Complexity	O(log n)	O(n)
Stability	Not Stable ❌	Stable ✅
Preferred for	In-memory sorting	Large data or external sorting
Implementation	Faster in practice	Easier to parallelize

◇ 7. Difference between Heap Sort and Merge Sort

Feature	Heap Sort	Merge Sort
Data structure used	Binary Heap	Divide arrays recursively
Time Complexity	O(n log n)	O(n log n)
Space Complexity	O(1)	O(n)
Stability	Not Stable ❌	Stable ✅
Implementation	In-place	Needs extra array
Use case	Limited memory systems	When stability required

◇ 8. Difference between Insertion Sort and Selection Sort

Feature	Insertion Sort	Selection Sort
Working	Inserts element in correct position	Finds smallest and swaps to correct place
Number of Swaps	More (depends on data)	Less
Best Case	O(n) (when array nearly sorted)	$O(n^2)$
Stability	Stable ✅	Not Stable ❌
Adaptive	Yes	No
Practical use	Small or partially sorted lists	Small unsorted lists

◇ 9. Difference between Stable and Unstable Sorting

Feature	Stable Sort	Unstable Sort
Definition	Maintains the order of equal elements	May change order of equal elements
Example	Merge Sort, Insertion Sort, Bubble Sort	Quick Sort, Heap Sort, Selection Sort
Importance	Useful when order matters (like names, IDs)	Not important when elements are unique

◇ 10. Difference between Comparison Sort and Non-Comparison Sort

Feature	Comparison Sort	Non-Comparison Sort
Concept	Elements compared to decide order	Uses other properties (like digits, keys)
Examples	Quick, Merge, Heap, Bubble	Counting, Radix, Bucket
Complexity	O(n log n) minimum	O(n) possible
Data Type	Works on all data	Works only on integers or fixed-length keys